

스마트 주차장 빈자리 감지 및 차량 입차 제어 시스템

NUCLEO-F429ZI · uC/OS-III Term Project

정보컴퓨터공학부 202055575 이동근

차 례

1. 개요	1
2. 프로젝트 개요 및 필요성	2
3. RTOS 적용 효과	2
4. 시스템 구성	2
4.1 하드웨어 구성	2
4.2 task 구성과 우선순위	3
5. task 간 통신 구조	4
6. 구현 내용	5
6.1 HC-SR04 4채널 측정	5
6.2 단일 디스패처와 센서 판정	5
6.3 예약과 예약 타임아웃	5
6.4 차단기 제어와 입차·만차 분기	5
6.5 표시와 로그	6
7. 동작 시나리오	6
8. 결론 및 토의	7
8.1 겪은 문제와 해결	7
8.2 설계 회고	8

1. 개요

임베디드 시스템은 특정 기능 하나를 정해진 시간 안에 반복해서 처리하는 전용 시스템이다. 센서를 읽고 모터나 표시 장치를 제어하는 일을 제때 해야 하는데, 이것 단일 **while** 루프로만 짜면 한 작업이 늦어질 때 나머지 반응까지 같이 밀린다.

그래서 RTOS를 쓴다. 기능을 독립적인 task로 나누고 우선순위 기반 선점형 스케줄링으로 급한 일을 먼저 처리한다. 본 프로젝트는 uC/OS-III를 써서 입력·판단·출력을 서로 다른 task로 나누고, task끼리는 queue와 semaphore로만 통신하도록 설계했다.

2. 프로젝트 개요 및 필요성

소규모 주차장에선 운전자가 직접 빈자리를 찾아 돌아다녀야 하고, 만차인데도 차가 들어오려다 입구가 막히곤 한다. 빈자리가 입구에서 바로 안내되고 자리가 있을 때만 차단기가 열린다면 이런 혼잡을 줄일 수 있다.

이 상황을 소형 모형 주차장으로 단순화했다. 주차칸 4개를 초음파 센서로 감지하고, 입구 적외선 센서가 차량을 감지하면 빈자리가 있을 때만 서보 차단기를 열었다가 자동으로 닫는다. 만차면 차단기를 안 열고 부저·LED로 알린다. 출차는 버튼으로 이벤트만 발생시키고, 빈자리 수는 초음파 측정으로만 갱신한다. 상태는 I2C LCD와 RGB LED로 표시하고 모든 이벤트는 USART 로그로 남긴다.

실제 차체를 만드는 게 목적이 아니라, “여러 입력이 동시에 들어오고 그에 따라 여러 출력을 제어해야 하는 상황”을 task로 어떻게 나누는지를 보이는 데 중점을 뒀다.

3. RTOS 적용 효과

이 시스템을 단일 루프로 짜면 문제가 바로 보인다. 초음파는 trigger 후 echo를 기다려야 하는데, 그 사이 루프가 멈춰 있으면 입구 차량이나 출차 버튼에 반응하지 못한다. LCD·USART 출력이 길어지면 급한 차단기 제어가 뒤로 밀리고, 상태를 전역 변수로 들고 있으면 값이 꼬일 수 있다.

단일 루프에서 어려운 점	uC/OS-III에서 해결되는 방식
echo 대기 중 다른 입력을 못 받음	센서 측정을 별도 task가 맡고 대기는 OSTimeDly 로 양보
긴 출력 작업이 급한 제어를 밀어냄	관리·차단기 task는 높은 우선순위, 표시·로그는 낮게
전역 변수 공유로 상태 추적이 어려움	상태를 한 task만 소유, 나머지는 queue로 요청만
이벤트와 주기 작업이 섞임	주기 task(센서)와 이벤트 task(버튼·입구)를 분리

핵심은 부품을 많이 붙이는 게 아니라 동시에 일어나는 일을 task로 잘 나눠 서로 방해하지 않게 하는 거라고 봤다.

4. 시스템 구성

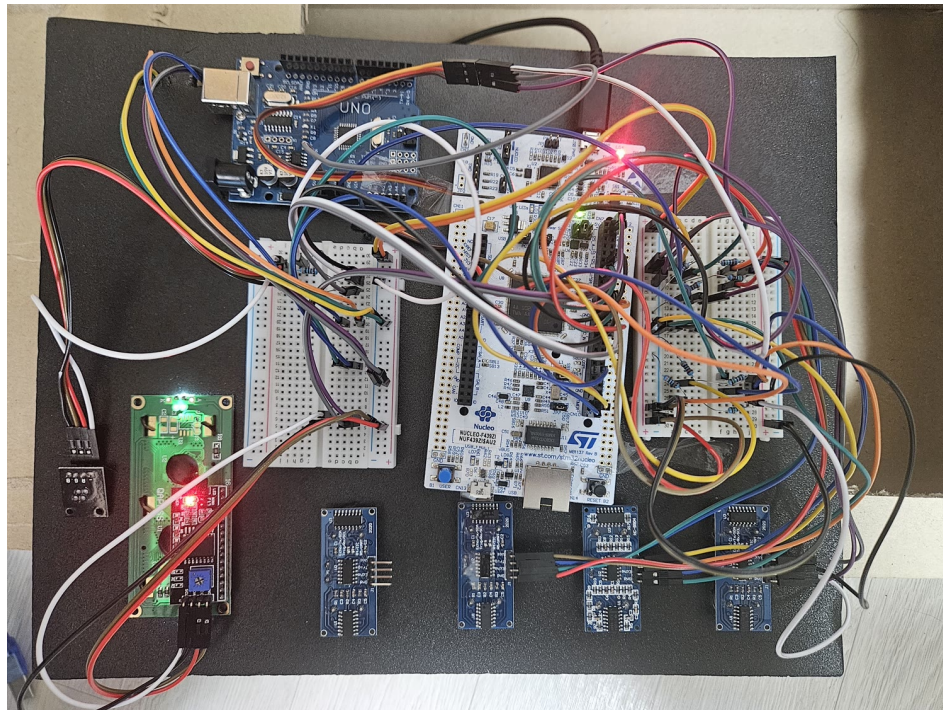
4.1 하드웨어 구성

장치	수량	역할	연결
HC-SR04 초음파	4	각 주차칸 점유 감지	TIM2 input capture (echo), GPIO (trig)
적외선 센서	1	입구 차량 접근 감지	GPIO
서보모터	1	입구 차단기 개폐	TIM3 PWM (50Hz)
1602 I2C LCD	1	빈자리·차단기 표시	I2C1
RGB LED	1	가능/예약/만차 색 표시	GPIO 3핀
부저	1	만차 경고음	GPIO
버튼	1	출차 이벤트	EXTI

초음파 4개의 echo는 TIM2 네 채널에, trig는 GPIO 출력으로 연결했다. 서보는 PA6(TIM3_CH1), LCD는 PB8/PB9, RGB LED는 PF13~15, 출차 버튼은 보드 USER 버튼(PC13)을 썼다.

센서	TRIG	ECHO	TIM2 채널
s0	PE0	PA0	CH1
s1	PE11	PA3	CH4
s2	PE9	PB10	CH3
s3	PE13	PB3	CH2

배선에서 신경 쓴 점이 몇 가지 있다. HC-SR04의 echo는 5V로 나와서 STM32 입력 핀 보호를 위해 $1k\Omega \cdot 2k\Omega$ 저항으로 분압을 걸었다. PA1·PA2는 보드 이더넷(RMII)에 물려 있어 못 쓰기 때문에 echo를 그 핀들을 피해 배치했다. 서보는 움직일 때 전류가 커서 외부 5V로 구동하고 GND만 보드와 공통으로 연결했다. 이때 외부 5V는 마침 가지고 있던 아두이노 보드로 공급했는데, 아두이노에는 아무 코드도 올리지 않고 5V 핀을 그냥 전원 공급용으로만 썼다.



전체 배선 (위쪽 아두이노 UNO는 서보 외부 5V 공급 전용, 가운데 NUCLEO-F429ZI, 아래 HC-SR04 4개)

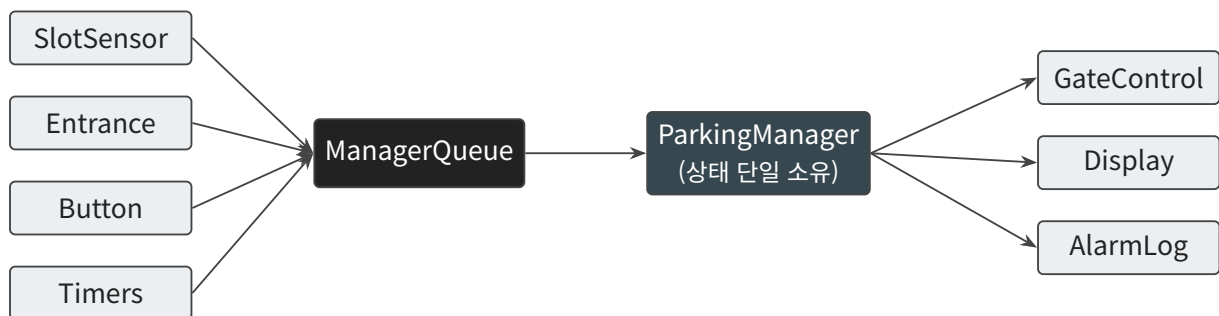
4.2 task 구성과 우선순위

Start Task를 빼고 7개의 응용 task로 나눴다(값이 작을수록 우선순위 높음).

task	우선순위	역할
ParkingManagerTask	4	상태 판단·빈자리 계산 (단일 소유자)
GateControlTask	5	서보 차단기 개폐
SlotSensorTask	6	초음파 4칸 순차 측정
EntranceTask	7	입구 차량 접근 감지
ButtonTask	8	출차 버튼 디바운스

task	우선순위	역할
DisplayTask	9	LCD·RGB LED 갱신
AlarmLogTask	10	부저·USART 로그

상태를 판단하고 차단기를 제어하는 task를 제일 높게, 조금 늦어도 되는 표시·로그를 제일 낮게 뒀다. 제일 신경 쓴 설계는 **단일 디스패처** 구조다. 모든 입력을 **ManagerQueue** 하나로 모으고 주차장 상태는 **ParkingManagerTask**만 가진다. 다른 task는 상태를 직접 건드리지 않고 queue로 요청만 보낸다. 상태를 바꾸는 코드가 한 곳에 모여 race condition을 아예 피할 수 있다.



5. task 간 통신 구조

전역 변수 없이 message queue, semaphore, software timer로만 통신을 구성했다.

객체	종류	post → pend	내용
ManagerQueue	Queue	센서·입구·버튼·타이머 → Manager	type + 데이터
GateCommandQueue	Queue	Manager → Gate	GATE_OPEN/CLOSE/DENY
DisplayQueue	Queue	Manager → Display	빈자리·칸 상태·게이트
LogQueue	Queue	Manager → Log	이벤트 코드
ButtonSem	Semaphore	버튼 ISR → Button	눌림 신호
GateTimer / ReserveTimer[4]	Timer	만료 콜백	자동 닫힘 / 예약 만료

queue로 구조체를 넘길 때 데이터 수명이 문제다. **OSQPost()**에 지역 변수 주소를 넘기면 함수가 끝난 뒤 그 주소를 믿을 수 없다. HW#6에서 겪었던 부분이라, 이번엔 메시지용 메모리 풀을 만들어 두고 블록을 받아 채운 뒤 포인터만 queue에 넣고, 받는 쪽이 처리 후 반납하게 했다.

```

p_msg = IPC_MsgAlloc();           /* 풀에서 블록 확보 */
if (p_msg != (MANAGER_MSG *)0) {
    p_msg->type      = MSG_SENSOR;
    p_msg->slot_id   = slot;
    p_msg->distance_mm = dist;
    IPC_PostManager(p_msg);        /* 포인터만 전달 */
}

```

반면 `GATE_CMD`나 `LOG_EVENT`처럼 작은 enum은 굳이 풀을 쓸 필요가 없어서 값을 포인터 자리에 그대로 인코딩해 보냈다. software timer도 만료 시 콜백에서 곧장 상태를 바꾸지 않고 `MSG_GATE_TIMEOUT/MSG_RESERVE_TIMEOUT` 메시지를 `ManagerQueue`로 보낸다. 시간으로 생기는 이벤트도 다른 입력과 똑같이 단일 디스패처를 거치게 하기 위해서다.

6. 구현 내용

6.1 HC-SR04 4채널 측정

초음파 측정은 echo 펄스 폭을 재는 일이다. delay로 재면 그 시간 동안 CPU가 묶이니까, TIM2 input capture 인터럽트로 상승·하강 에지 타임스탬프를 잡아 폭을 계산했다. TIM2는 1μs마다 올라가고, 폭에 음속을 적용해 거리(mm)로 환산한다. TIM2가 채널이 4개라 echo 4개를 한 타이머에 다 연결했고, ISR은 어느 채널인지 배열로 돌려 처리해서 센서를 늘려도 구조가 유지된다.

`SlotSensorTask`는 센서를 한 번에 하나씩 trigger한다. 4개를 동시에 쏘면 서로의 반향을 잡아 간섭이 생기길래, 한 칸을 쏘고 echo가 돌아올 시간을 `OSTimeDly`로 기다린 뒤 다음 칸으로 넘어간다. 측정 실패는 `INVALID`로 정규화해서 관리 task가 떨림으로 오인하지 않게 했다.

6.2 단일 디스패처와 센서 판정

`ParkingManagerTask`는 상태(칸 배열·빈자리 수·차단기 여부)를 혼자 가져서 lock이 필요 없다. `ManagerQueue`에서 메시지를 꺼내 type별로 분기하고, 처리할 때마다 상태 스냅샷을 `DisplayQueue`로 보낸다.

```
switch (p_msg->type) {
    case MSG_SENSOR:      Manager_OnSensor(p_msg->slot_id, p_msg->distance_mm); break;
    case MSG_ENTRANCE:    Manager_OnEntrance();                               break;
    case MSG_EXIT:        Manager_OnExit();                                   break;
    case MSG_GATE_TIMEOUT: Manager_OnGateTimeout();                           break;
    case MSG_RESERVE_TIMEOUT: Manager_OnReserveTimeout(p_msg->slot_id);       break;
}
```

처음엔 거리가 기준보다 작으면 바로 점유로 봤는데, 측정값이 한두 번씩 튀어서 상태가 깜빡거렸다. 그래서 같은 판정이 연속 3회 나왔을 때만 상태를 바꾸는 hysteresis를 넣었다.

6.3 예약과 예약 타임아웃

차단기를 열어도 차가 칸에 들어가 점유로 확정되기까진 시간이 걸린다. 그 사이 빈자리 수가 그대로면 마지막 한 자리에 두 대가 들어올 수 있다. 그래서 입차를 허용하는 순간 빈칸 하나를 `RESERVED`로 잠가 빈자리 수에서 빼고, 센서가 점유를 확정하면 `OCCUPIED`로 바꾼다.

차단기를 열었는데 차가 끝내 안 들어오면 칸이 계속 잠긴다. 그래서 입차 허용과 동시에 `Reserve-Timer`를 켜고, 10초 안에 점유가 확정 안 되면 타이머가 만료되며 칸을 다시 `EMPTY`로 되돌린다. 이 만료도 queue 메시지로 들어와 처리된다.

6.4 차단기 제어와 입차·만차 분기

입구 차량이 감지되면 빈자리 수를 확인한다. 있으면 빈칸을 `RESERVED`로 잠그고 예약 타이머를 켜 뒤 `GATE_OPEN`을 보내고, 5초 뒤 자동으로 닫는 `GateTimer`를 켜다. 없으면 차단기는 그대로 두고 `GATE_DENY`와 만차 로그만 보낸다.

```

if (FreeCount > 0u) {
    Manager_SendGate(GATE_OPEN);
    OSTmrStart(&GateTimer, &err);
    IPC_PostLog(LOG_GATE_OPEN);
} else {
    Manager_SendGate(GATE_DENY);
    IPC_PostLog(LOG_FULL_DENIED);
}

```

GateControlTask는 명령을 받아 서보 PWM 펄스 폭만 바꾼다. 서보는 TIM3로 50Hz PWM을 내고, 닫힘 2.0ms·열림 1.0ms 펄스로 각도를 제어한다.

6.5 표시와 로그

LCD·RGB LED는 **DisplayTask**만, USART·부저는 **AlarmLogTask**만 건드린다. 공유 자원을 mutex로 보호하는 대신 한 task만 소유하게 해서 충돌을 없앴다. LCD엔 **FREE:n/N**과 각 칸 상태(E/O/R), 차단기 상태를 표시한다.

처음엔 메시지가 올 때마다 LCD를 다시 그렸는데 센서가 200ms마다 돌아 화면이 깜빡거렸다. 그래서 직전 출력 줄과 LED 색을 기억해 두고 진짜 바뀐 줄만 다시 그리게 했다. LED 색은 **RESERVED**(노랑)를 먼저 확인하고, 예약이 하나도 없으면서 빈자리도 없을 때만 빨강으로 처리한다. 처음엔 빈자리 수부터 보다가 두 칸이 다 예약일 때 빨강으로 나와서 순서를 바로잡았다.

USART 출력은 전부 **AlarmLogTask** 한 곳으로 모았다. 여러 task가 각자 찍으면 출력이 섞이기 때문이다. 로그만 봐도 어느 이벤트가 처리됐는지 따라갈 수 있다.

```

[EVENT] entrance_detected
[GATE] open / close
[BUTTON] exit_event_logged
[ALARM] parking_full entrance_denied
[EVENT] reservation_expired

```

7. 동작 시나리오

단계	동작	확인
1	부팅	FREE:4/4 S:EEEE , 게이트 CLOSED
2	접근 (빈자리 있음)	빈칸 RESERVED, OPEN, 5초 후 CLOSE
3	점유 확정	칸 OCCUPIED, 빈자리 감소
4	만차에서 접근	차단기 유지, 부저 + entrance_denied
5	출차 버튼	exit_event_logged (상태는 직접 안 바꿈)
6	차량 제거	초음파가 EMPTY 판정, 빈자리 복구
7	입차 허용 후 미입차	10초 뒤 예약 만료, EMPTY 복귀

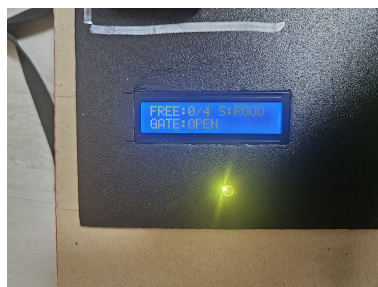
버튼은 출차 이벤트만 기록하고 칸 상태를 직접 비우지 않는다. 점유 판단 기준을 초음파 하나로 통일하기 위해서다.

출차 버튼은 따로 단 버튼이 제때 도착하지 않아서 보드에 내장된 USER 버튼(PC13)으로 대신 구현했

다. 패키징하면서 보드를 안쪽에 숨기다 보니 영상에는 이 버튼이 직접 보이지 않는데, 출차 이벤트가 처리되는 건 USART 로그의 `exit_event_logged`로 확인할 수 있다.



빈자리 (LED 초록)



예약·입차 중 (LED 노랑)



점유 (해당 칸 0)

USART 로그도 본문 예시와 동일하게 단계별로 찍힌다. 출구 버튼(`exit_event_logged`)도 여기서 확인할 수 있다.

```
[EVENT] entrance_detected
[GATE] open
[GATE] close
[EVENT] entrance_detected
[ALARM] parking_full entrance_denied
[EVENT] entrance_detected
[ALARM] parking_full entrance_denied
[EVENT] entrance_detected
[GATE] open
[GATE] close
[EVENT] reservation_expired
[BUTTON] exit_event_logged
```

- 시나리오 동작 영상: <https://www.youtube.com/watch?v=EjltmRGkgJg>
- USART 로그 출력 영상: https://www.youtube.com/watch?v=UNglfn_leFg

8. 결론 및 토의

8.1 겪은 문제와 해결

제일 오래 잡은 건 LCD에 글자가 안 뜨는 문제였다. 로그는 멀쩡한데 LCD만 부팅 배너에서 멈춰 있었다. USART 디버그를 단계별로 넣어 좀해보니, 표시 메시지 풀을 만드는 `OSMemCreate`가 `OS_ERR_MEM_INVALID_SIZE`로 실패하고 있었다. `OSMemCreate`가 블록 크기를 4바이트 배수로 요구하는데, 이 프로젝트가 `-fshort-enums`로 컴파일돼 enum이 1바이트가 되면서 구조체 크기가 안 맞은 거였다. 칸이 2개일 땐 우연히 통과했는데 4개로 늘리며 드러났다. 구조체에 `aligned(4)`를 붙여 해결했다.

서보도 처음엔 코드 문제인 줄 알았다. 명령은 가는데 차단기가 안 움직이고, 부저가 “딱딱” 소리를 냈다. 서보 전원을 빼니 소리가 멈췄다. 서보가 움직이는 순간 전류를 끌어쓰며 5V 레일이 출렁였고 그 리플이 부저에 나타난 거였다. 외부 5V로 서보를 구동하니 해결됐다.

핀 충돌도 있었다. echo를 PA1에 연결하려다 점퍼 곳을 핀이 안 보였는데, PA1·PA2가 이더넷(RMII)

에 매핑돼 있었다. echo를 이더넷과 상관없는 핀으로 옮기고 아두이노 호환 핀 위주로 배치를 다시 정리했다.

초음파 센서는 배선하다 하나를 태워 먹기도 했다. 정확한 순간을 보진 못했지만, 점퍼를 다시 꽂는 과정에서 VCC와 GND를 반대로 물린 게 원인이었던 것 같다. 한 번 전원을 잘못 넣으니 센서가 그대로 죽어버려서, 그 뒤로는 전원을 넣기 전에 +- 방향부터 다시 확인하는 습관이 생겼다.

8.2 설계 회고

앞의 문제들은 증상이 대개 “동작 안 함”으로 비슷했는데 원인은 정렬 규칙, 전원, 핀 매핑, 배선 실수로 제각각이었다. task와 통신 경로를 명확히 나눠둔 덕분에 USART 로그를 따라 어느 단계에서 흐름이 끊겼는지 좁힐 수 있었던 게 컸다. 반대로 센서를 태워 먹은 일은 코드로는 알 수 없는 부분이라, 하드웨어는 전원을 넣기 전 배선을 확인하는 게 결국 제일 빠른 디버깅이라는 걸 느꼈다.

단일 디스패처, 메모리 풀, 자원을 한 task만 소유하게 한 방식이 잘 맞았다. 상태를 바꾸는 코드가 한 곳뿐이고 LCD·서보·USART도 각각 한 task만 만지니 lock 거의 없이 경합이 안 생겼다. 주차칸 수를 상수 하나로 관리해서, 2칸 구조를 상수만 4로 바꿔 그대로 확장할 수 있었다.

이번 프로젝트로 여러 입력·출력을 task로 나누고 queue·semaphore·timer로 통신하면서 상태 머신과 타임아웃·예외 복구까지 직접 다뤄봤다. 단일 디스패처를 유지하는 한 새 입력을 추가해도 상태 관리 코드는 그대로 두고 메시지 type만 늘리면 된다.